

Projet Deep Reinforcement Learning

Évaluation comparative de onze algorithmes sur quatre environnements

Line World · Grid World · TicTacToe versus Random · Bobail

M2 — IABD — Deep Reinforcement Learning

Enseignant : Nicolas Vidal

Groupe

Omar Kaabi · Awatef B. · Viet Hoang

Sommaire

1. Objet et périmètre.....	3
1.1 Ce que le rapport établit.....	3
1.2 Organisation du code.....	3
2. Les environnements.....	3
2.1 Interface commune.....	3
2.2 Encodages retenus.....	4
2.3 Débit de simulation.....	4
3. Les onze agents.....	4
3.1 Famille valeur.....	4
3.2 Famille gradient de politique.....	5
3.3 PPO, style A2C.....	5
3.4 Agents de planification.....	6
4. Protocole expérimental.....	6
4.1 La politique obtenue, jamais la politique d'entraînement.....	6
4.2 Graines et reproductibilité.....	6
4.3 Choix des hyperparamètres.....	6
5. Résultats.....	8
5.1 Line World.....	8
5.2 Grid World.....	8
5.3 TicTacToe versus Random.....	9
5.4 Bobail contre un adversaire aléatoire.....	10
5.5 Bobail contre un adversaire heuristique.....	11
6. Observations critiques.....	13
7. Limites de l'étude.....	15
8. Conclusion.....	16
Annexe — Reproduire les résultats.....	17

1. Objet et périmètre

Ce rapport présente l'évaluation comparative de onze algorithmes d'apprentissage par renforcement, dont huit apprennent par descente de gradient sur un réseau de neurones, sur quatre environnements : les trois environnements de test imposés — Line World, Grid World et TicTacToe versus Random — et Bobail, retenu comme environnement libre.

La campagne totalise 165 entraînements indépendants et plus de 55 millions d'épisodes joués. Chaque configuration a été répétée sur trois graines, et chaque point de mesure porte sur la politique obtenue, évaluée sans exploration sur 200 parties.

1.1 Ce que le rapport établit

- **Un résultat sur le protocole** — trois des quatre environnements sont saturés : presque tous les agents y atteignent le score maximal, et le tableau ne discrimine rien.
- **Le classement final n'est pas un résultat** — à un million d'épisodes sur TicTacToe, les agents de la famille valeur et PPO sont tous entre 0,93 et 0,99, avec des écarts entre graines du même ordre que leurs différences.
- **La stabilité discrimine mieux que le score** — l'écart-type entre graines à 10 000 épisodes sépare nettement les algorithmes là où le score final les confond.
- **La recherche sans apprentissage a une limite mesurable** — contre un adversaire aléatoire, MCTS et Random Rollout atteignent le score maximal sans rien apprendre ; contre un adversaire heuristique, ils échouent, là où un agent entraîné réussit — et pour un coût par décision 2 400 fois inférieur.

1.2 Organisation du code

Le code est organisé autour d'un contrat unique : `src/envs/` expose une interface identique pour les quatre jeux, `src/agents/` contient les onze agents, et `src/train.py` les fait dialoguer sans qu'un agent sache jamais à quel jeu il joue. Un agent reçoit un vecteur d'état et un masque d'actions légales, il renvoie un numéro d'action.

2. Les environnements

2.1 Interface commune

Les quatre environnements exposent les mêmes méthodes. Deux points de conception méritent d'être signalés.

Le masque d'actions. La légalité d'une action est portée par `available_actions_mask()`, jamais par un changement de taille de l'espace d'actions. Au Bobail, moins de 5 % des 208 actions sont légales à un instant donné : sans masque, un agent consacrerait l'essentiel de son budget d'entraînement à apprendre la légalité plutôt que la stratégie. Les valeurs illégales sont forcées à -inf avant tout `argmax` et avant le maximum de la cible temporelle ; les logits de politique reçoivent -1e9 avant le `softmax`, une valeur finie choisie pour ne pas produire de NaN dans le gradient.

L'adversaire est intérieur à `step()`. Pour TicTacToe et Bobail, le coup adverse est joué à l'intérieur de la transition. Du point de vue de l'agent, l'environnement reste un problème à un seul joueur, simplement stochastique. La récompense d'un pas est donc la différence du score cumulé avant et après l'appel.

2.2 Encodages retenus

Environnement	État	Actions	Encodage
Line World	5	2	one-hot de la position
Grid World	25	4	one-hot de la case
TicTacToe	27	9	3 plans de 9 cases ; action = case jouée
Bobail	77	208	3 plans de 25 cases + phase en one-hot

Deux choix d'encodage sont à justifier.

Bobail, espace d'actions. Un tour est un couple : déplacer le bobail, puis faire glisser un pion. Encodé d'un seul bloc, cela produit $8 \times 200 = 1\,600$ actions dont une fraction infime est légale. Le tour est donc découpé en deux demi-coups, signalés par un drapeau de phase présent dans l'état, ce qui ramène l'espace à $8 + 25 \times 8 = 208$ actions. Contrepartie assumée : le crédit d'une victoire doit remonter sur deux fois plus de pas, ce qui allonge l'horizon effectif.

Bobail, désignation du pion. Une action désigne le pion par sa **case de départ**, et non par un numéro d'ordre. La raison est que la case figure dans l'entrée du réseau — les plans de 25 cases — alors qu'un numéro de pion n'y figure pas. Un agent à qui l'on demande de déplacer « le pion numéro 2 » devrait désigner une information qu'il ne reçoit jamais.

2.3 Débit de simulation

Mesuré en jeu totalement aléatoire. Ces chiffres bornent ce qui est atteignable et justifient les paliers effectivement mesurés.

Environnement	Parties/s	Pas/partie
Line World	63 064	4,0
Grid World	4 430	52,9
TicTacToe	19 073	4,2
Bobail	2 639	10,3

Grid World coûte dix fois plus par épisode que TicTacToe, ses parties durant une cinquantaine de coups contre quatre. C'est ce facteur qui rend le palier d'un million inaccessible sur cet environnement.

3. Les onze agents

Les agents se rangent en trois familles. À l'intérieur d'une famille, chacun n'ajoute qu'un mécanisme au précédent, ce qui permet d'attribuer un effet observé à une cause précise.

3.1 Famille valeur

Deep Q-Learning apprend une valeur d'action et en déduit la politique par $\arg\max$. La cible temporelle est $r + \gamma \cdot \max_a Q(s', a)$, le maximum étant pris sur les seules actions légales de s' . Cette cible est bootstrappée : elle dépend du réseau que l'on modifie, ce qui revient à poursuivre une cible mouvante.

Double DQN ajoute deux mécanismes. Un réseau cible gelé, recopié tous les 500 apprentissages, stabilise la cible. Et le choix de l'action est découplé de son évaluation : $a^* = \operatorname{argmax} Q_{\text{en-ligne}}(s', a')$, puis $y = r + \gamma \cdot Q_{\text{cible}}(s', a^*)$. Ce découplage corrige un biais systématique — les erreurs d'estimation étant aléatoires, le maximum tombe préférentiellement sur les actions surestimées, et ce biais s'accumule.

Double DQN + Experience Replay n'apprend plus sur la transition courante mais l'empile, et tire un lot uniforme toutes les quatre transitions. Deux effets : la corrélation temporelle entre transitions consécutives est brisée, et une expérience rare — une victoire — continue d'être retirée pendant des milliers de pas.

Double DQN + Prioritized Replay tire proportionnellement à $|\text{erreur TD}| + \epsilon$, élevé à la puissance $\alpha = 0,6$, via un arbre de sommes en $O(\log n)$. Le tirage non uniforme biaisant l'espérance du gradient, chaque élément du lot est pondéré par $w = (N \cdot P(i))^{(-\beta)}$, avec β annelé de 0,4 vers 1,0. Cette pondération n'est pas optionnelle : sans elle, l'agent apprend plus vite au début puis converge vers une politique biaisée.

3.2 Famille gradient de politique

REINFORCE paramètre directement la politique et remonte le gradient qui augmente la probabilité des actions ayant mené à un bon retour. La mise à jour n'a lieu qu'en fin d'épisode, les retours G_t exigeant de connaître toutes les récompenses suivantes. C'est une méthode Monte Carlo, d'où sa variance élevée : deux épisodes identiques au début mais différents à la fin produisent des gradients opposés sur les mêmes premiers pas. L'exploration ne vient pas d'un ϵ ajouté mais de l'échantillonnage dans la politique, qui est stochastique par construction.

REINFORCE + baseline soustrait aux retours leur moyenne. L'opération est légitime parce que soustraire une quantité indépendante de l'action ne change pas l'espérance du gradient — $E[\nabla \log \pi(a|s)] = 0$ — mais en réduit la variance. Concrètement : sans baseline, si tous les retours d'un épisode sont positifs, toutes les actions voient leur probabilité augmenter, y compris les mauvaises ; avec, celles situées sous la moyenne sont explicitement découragées.

REINFORCE + critique remplace cette baseline constante par un réseau $V(s)$ entraîné en parallèle à prédire le retour. L'avantage devient $G_t - V(s_t)$. L'avantage est détaché du graphe : sans cela, le gradient de la politique remonterait dans le critique et l'entraînerait à minimiser l'avantage plutôt qu'à prédire le retour.

3.3 PPO, style A2C

PPO répond à un défaut de REINFORCE : rien n'empêche une mise à jour de déplacer massivement la politique, et comme la méthode est on-policy, les données suivantes sont collectées par la politique dégradée — l'effondrement s'auto-entretient. PPO borne le déplacement en optimisant $\min(r \cdot A, \text{clip}(r, 1-\epsilon, 1+\epsilon) \cdot A)$, où r est le rapport entre la politique courante et celle qui a collecté les données. Le minimum annule tout gain à dépasser la borne, ce qui permet plusieurs passes sur le même lot sans divergence. L'avantage est calculé par $\text{GAE}(\lambda = 0,95)$.

Le masque de l'état est stocké avec la transition. Les log-probabilités sont recalculées à chaque passe ; si le masque différait de celui de la collecte, les rapports seraient faux. Le défaut serait invisible : l'entraînement tournerait, simplement moins bien.

L'horizon a été réduit de 2 048 à 256 pas. Nos parties durent de 2 à 10 pas ; avec l'horizon usuel, 10 000 épisodes ne produiraient qu'une poignée de mises à jour.

3.4 Agents de planification

Random Rollout évalue chaque action légale en clonant l'environnement, jouant le coup, puis terminant la partie au hasard vingt fois ; il retient l'action au meilleur score final moyen.

MCTS (UCT) enchaîne sélection par UCB1, expansion, simulation aléatoire et rétropropagation, 200 fois par coup. L'implémentation est en boucle ouverte : les nœuds ne mémorisent pas l'état. L'environnement étant stochastique de notre point de vue — l'adversaire joue dans `step()` — chaque itération repart d'un clone de la racine et rejoue le chemin d'actions, si bien que les statistiques d'un nœud moyennent sur les réponses possibles de l'adversaire. L'action jouée est la plus visitée et non la mieux notée : une action peu visitée peut afficher une moyenne flatteuse obtenue sur deux simulations chanceuses.

Ces deux agents ne possèdent pas de réseau et ne s'entraînent pas. Ils sont donc évalués aux mêmes paliers que les autres, mais leur score n'évolue pas avec le nombre d'épisodes.

4. Protocole expérimental

4.1 La politique obtenue, jamais la politique d'entraînement

Toutes les métriques portent sur la politique obtenue. La fonction `evaluate()` gèle l'agent en mode glouton — $\epsilon = 0$, `argmax` au lieu d'échantillonnage — et joue 200 parties. Un score mesuré avec 10 % d'exploration résiduelle ne mesurerait pas la politique apprise.

4.2 Graines et reproductibilité

Chaque configuration est entraînée sur trois graines. Chaque partie d'évaluation reçoit en outre sa propre graine, décalée loin des graines d'entraînement, afin de mesurer sur des parties non vues tout en restant reproductible.

Deux précautions conditionnent la validité de ces mesures.

La graine de l'adversaire doit être fixée. Le paramètre `seed` passé à un agent ne pilote que le hasard de celui-ci. Si l'environnement crée son adversaire sans graine, deux entraînements identiques produisent des politiques différentes et aucun résultat n'est rejouable.

La graine d'évaluation doit varier d'une partie à l'autre. Une fabrique qui réutiliserait la même graine rejouerait N fois la même partie : la moyenne porterait sur un unique tirage et l'écart-type serait nul.

Nous avons vérifié la reproductibilité obtenue en relançant un entraînement depuis sa seule graine : il a reproduit les valeurs attendues au centième, sur deux paliers indépendants.

4.3 Choix des hyperparamètres

L'architecture du réseau est identique pour tous les agents neuronaux — deux couches cachées de 128 unités. C'est ce qui rend la comparaison entre algorithmes honnête : l'écart observé ne peut pas être attribué à une capacité différente.

Paramètre	Valeur	Rôle et justification
hidden	[128, 128]	identique partout, pour isoler l'effet de

		l'algorithme
gamma	0,99	élevé : la récompense n'arrive qu'en fin de partie
learning_rate	0,001 / 0,0005	abaissé pour les agents à rejou, qui font plus de mises à jour
epsilon	1,0 → 0,05	exploration de la famille valeur
epsilon_decay	0,9995	plancher atteint vers 6 000 épisodes
target_update	500	période de recopie du réseau cible
batch_size	64	taille du lot tiré dans la mémoire
buffer_capacity	100 000	capacité de la mémoire de rejou
warmup	1 000	évite d'apprendre sur une mémoire quasi vide
train_every	4	une mise à jour toutes les 4 transitions
alpha	0,6	intensité de la priorisation ; 0 redonne l'uniforme
beta	0,4 → 1,0	correction du biais d'importance, annulée
clip_epsilon	0,2	largeur de la zone de confiance de PPO
gae_lambda	0,95	curseur biais / variance de l'avantage
horizon	256	réduit de 2 048 : nos parties sont très courtes
epochs	4	passes par rollout, permises par le clipping
entropy_coefficient	0,01	décourage une politique trop vite déterministe
iterations (MCTS)	200	simulations par coup ; fixe son coût
exploration (UCB1)	1,41	$\sqrt{2}$, la valeur théorique
rollouts_per_action	20	simulations par action pour Random Rollout

5. Résultats

Score moyen de la politique obtenue, sur trois graines. La valeur après $\pm$ est l'écart entre la meilleure et la moins bonne graine. Le score vaut +1 pour une victoire, -1 pour une défaite, 0 pour un nul ou une limite de pas atteinte.

5.1 Line World

Agent	1 000 ép.	10 000 ép.	100 000 ép.	1 000 000 ép.
Random	$+0,02 \pm 0,17$	$+0,02 \pm 0,06$	$+0,05 \pm 0,23$	$+0,02 \pm 0,05$
Deep Q-Learning	$+0,33 \pm 1,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
Double Deep Q-Learning	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
Double DQN + Experience Replay	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
Double DQN + Prioritized Replay	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
REINFORCE	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
REINFORCE + baseline	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$-0,33 \pm 2,00$
REINFORCE + critique	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
PPO (A2C style)	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
Random Rollout	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
MCTS (UCT)	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$

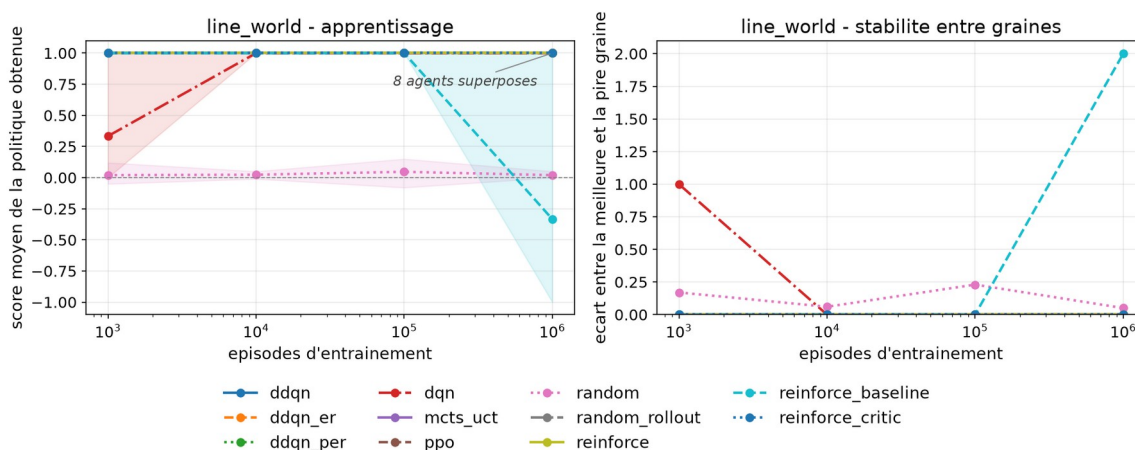


Figure 1 — Line World. À gauche l'apprentissage, à droite l'écart entre graines.

Environnement saturé. Dix agents sur onze atteignent +1,00 dès 1 000 épisodes ; seul Deep Q-Learning y arrive à 10 000. L'anomalie notable est l'effondrement de REINFORCE + baseline au palier d'un million, détaillé en 6.6.

5.2 Grid World

Agent	1 000 ép.	10 000 ép.	100 000 ép.
Random	$-0,15 \pm 0,20$	$-0,08 \pm 0,12$	$-0,10 \pm 0,18$

Deep Q-Learning	$+0,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
Double Deep Q-Learning	$+0,67 \pm 1,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
Double DQN + Experience Replay	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
Double DQN + Prioritized Replay	$+0,67 \pm 1,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
REINFORCE	$+0,00 \pm 0,00$	$+0,67 \pm 1,00$	$+0,00 \pm 2,00$
REINFORCE + baseline	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+0,67 \pm 1,00$
REINFORCE + critique	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
PPO (A2C style)	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
Random Rollout	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
MCTS (UCT)	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$

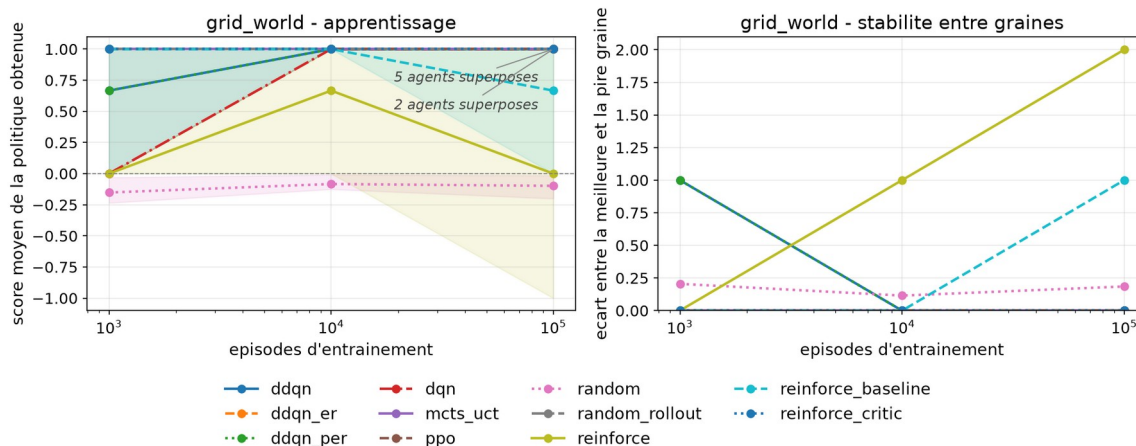


Figure 2 — Grid World. Le palier d'un million n'a pas été atteint (voir 7.1).

Environnement également saturé à partir de 10 000 épisodes. Il met toutefois en évidence l'instabilité de REINFORCE, qui oscille entre 0,00 et 0,67 sans se stabiliser, quand sa variante à critique atteint et conserve +1,00.

5.3 TicTacToe versus Random

Agent	1 000 ép.	10 000 ép.	100 000 ép.	1 000 000 ép.
Random	$+0,32 \pm 0,18$	$+0,27 \pm 0,04$	$+0,28 \pm 0,16$	$+0,33 \pm 0,04$
Deep Q-Learning	$+0,79 \pm 0,10$	$+0,90 \pm 0,04$	$+0,95 \pm 0,09$	$+0,98 \pm 0,06$
Double Deep Q-Learning	$+0,74 \pm 0,16$	$+0,94 \pm 0,08$	$+0,96 \pm 0,01$	$+0,93 \pm 0,01$
Double DQN + Experience Replay	$+0,86 \pm 0,06$	$+0,94 \pm 0,03$	$+0,99 \pm 0,02$	$+0,96 \pm 0,03$
Double DQN + Prioritized Replay	$+0,93 \pm 0,07$	$+0,95 \pm 0,05$	$+0,97 \pm 0,03$	$+0,99 \pm 0,01$
REINFORCE	$+0,72 \pm 0,23$	$+0,78 \pm 0,06$	$+0,79 \pm 0,15$	$+0,82 \pm 0,10$
REINFORCE + baseline	$+0,73 \pm 0,26$	$+0,80 \pm 0,06$	$+0,80 \pm 0,04$	$+0,84 \pm 0,09$

REINFORCE + critique	$+0,84 \pm 0,05$	$+0,85 \pm 0,30$	$+0,78 \pm 0,17$	$+0,81 \pm 0,07$
PPO (A2C style)	$+0,70 \pm 0,19$	$+0,92 \pm 0,04$	$+0,98 \pm 0,02$	$+0,98 \pm 0,03$
Random Rollout	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
MCTS (UCT)	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$

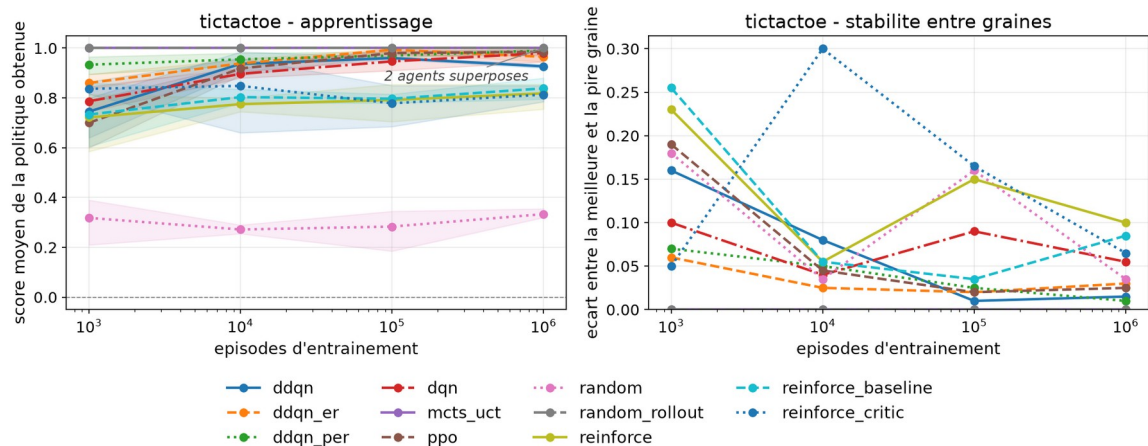


Figure 3 — TicTacToe. Le seul des trois environnements de test qui discrimine les agents.

Le seul environnement de test qui sépare les agents : ils s'étalent de 0,82 à 1,00. Trois observations.

- **Le plancher est franchi** — les onze agents, sauf Random, dépassent nettement l'agent aléatoire (+0,33). Le protocole mesure donc bien un apprentissage.
- **La famille gradient de politique plafonne** — REINFORCE, sa variante à baseline et sa variante à critique restent entre 0,81 et 0,84, et ne progressent plus entre 10 000 et 1 000 000 d'épisodes.
- **PPO échappe à ce plafond** — PPO atteint 0,98, à égalité avec les meilleurs agents de la famille valeur, alors qu'il appartient à la même famille que REINFORCE. La différence est le rapport clippé.

5.4 Bobail contre un adversaire aléatoire

Agent	1 000 ép.	10 000 ép.	100 000 ép.
Random	$+0,02 \pm 0,05$	$-0,05 \pm 0,20$	$-0,02 \pm 0,06$
Deep Q-Learning	$+0,45 \pm 1,16$	$+1,00 \pm 0,01$	$+1,00 \pm 0,00$
Double Deep Q-Learning	$+0,97 \pm 0,08$	$+0,98 \pm 0,02$	$+1,00 \pm 0,01$
Double DQN + Experience Replay	$+0,99 \pm 0,01$	$+0,71 \pm 0,54$	$+0,99 \pm 0,01$
Double DQN + Prioritized Replay	$+0,99 \pm 0,03$	$+0,60 \pm 0,99$	$+0,97 \pm 0,02$
REINFORCE	$+1,00 \pm 0,01$	$+0,99 \pm 0,03$	$+1,00 \pm 0,00$
REINFORCE + baseline	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,01$
REINFORCE + critique	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+0,99 \pm 0,00$
PPO (A2C style)	$+1,00 \pm 0,01$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
Random Rollout	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$
MCTS (UCT)	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$	$+1,00 \pm 0,00$

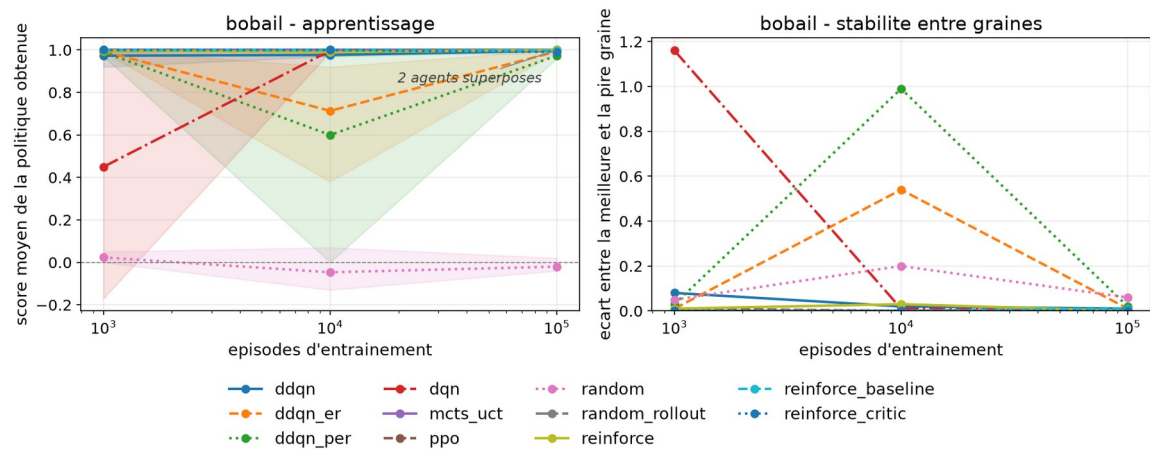


Figure 4 — Bobail, adversaire aléatoire. Tous les agents apprenants convergent vers +1,00 : l'environnement ne discrimine pas.

Dix agents sur onze atteignent ou approchent +1,00 à 100 000 épisodes. Cet adversaire ne permet donc aucune comparaison — constat qui a motivé la campagne suivante.

5.5 Bobail contre un adversaire heuristique

Même environnement, même code, même protocole : seul l'adversaire change. Il applique trois priorités — jouer un coup gagnant s'il existe, sinon écarter les coups qui offrent la victoire à l'agent, sinon pousser le bobail vers son camp.

Agent	1 000 ép.	10 000 ép.	100 000 ép.
Random	-1,00 ± 0,01	-1,00 ± 0,01	-1,00 ± 0,01
Deep Q-Learning	-0,72 ± 0,34	-0,85 ± 0,27	-0,63 ± 1,06
Double Deep Q-Learning	-0,89 ± 0,12	+0,72 ± 0,46	+0,80 ± 0,21
Double DQN + Experience Replay	-0,85 ± 0,06	+0,56 ± 0,01	+0,98 ± 0,03
Double DQN + Prioritized Replay	-0,74 ± 0,20	-0,33 ± 0,32	+0,97 ± 0,03
REINFORCE	-0,02 ± 0,35	+0,61 ± 0,12	+0,57 ± 0,16
REINFORCE + baseline	-0,45 ± 0,28	-0,43 ± 1,39	+0,09 ± 0,93
REINFORCE + critique	+0,32 ± 0,55	+0,40 ± 0,36	+0,86 ± 0,17
PPO (A2C style)	+0,67 ± 0,30	+0,97 ± 0,06	+1,00 ± 0,00
Random Rollout	-0,64 ± 0,12	-0,69 ± 0,11	-0,69 ± 0,05
MCTS (UCT)	-0,51 ± 0,09	-0,53 ± 0,03	-0,52 ± 0,04

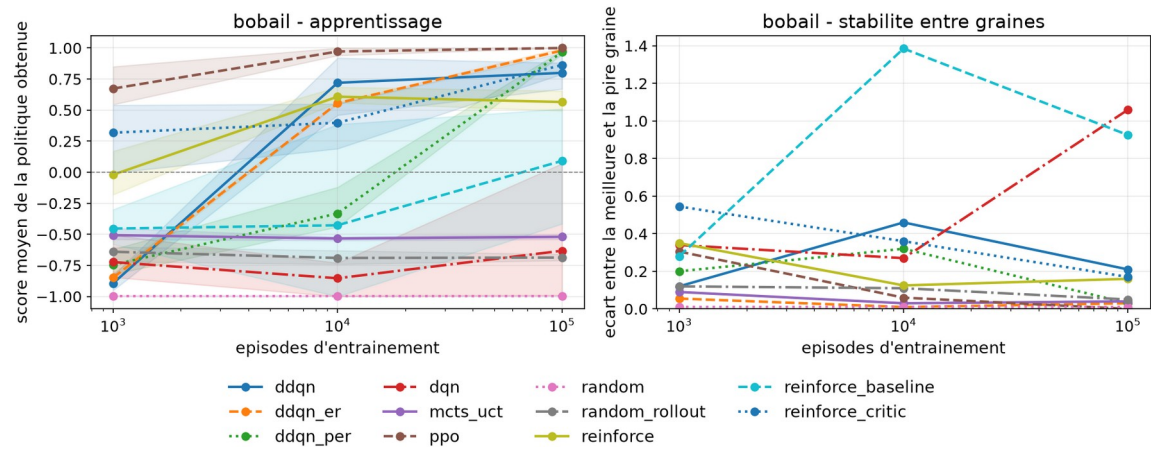


Figure 5 — Bobail, adversaire heuristique. Les agents s'étalent de $-1,00$ à $+1,00$: c'est l'environnement le plus discriminant de l'étude.

C'est le résultat central de ce rapport. Il est développé en 6.4 et 6.5.

6. Observations critiques

6.1 Trois environnements sur quatre sont saturés

Line World, Grid World et Bobail contre un adversaire aléatoire placent presque tous les agents à +1,00. Un tableau où tout le monde obtient le score maximal ne compare rien : il indique seulement que la tâche est trop facile pour l'ensemble des méthodes testées.

Ce n'est pas un défaut de notre travail mais la fonction de ces environnements : le sujet les désigne comme environnements de test. Leur rôle est de vérifier qu'un algorithme apprend, non de les classer. Nous en tirons une conséquence méthodologique : toute conclusion comparative doit reposer sur TicTacToe et sur Bobail contre l'adversaire heuristique.

6.2 Le classement final n'est pas un résultat

Sur TicTacToe à un million d'épisodes, les quatre agents de la famille valeur et PPO s'échelonnent de 0,93 à 0,99, avec des écarts entre graines de 0,01 à 0,06. Les différences sont du même ordre que le bruit. Annoncer un vainqueur reviendrait à surinterpréter.

La conclusion défendable est l'inverse : sur une tâche suffisamment simple, ces cinq algorithmes convergent vers le même niveau. Ce qui les distingue se lit ailleurs.

6.3 La stabilité entre graines discrimine mieux que le score

À 10 000 épisodes sur TicTacToe :

Agent	Score	Écart entre graines
Deep Q-Learning	0,832	0,156
Double Deep Q-Learning	0,940	0,007
Double DQN + Replay	0,901	0,079
Double DQN + Prioritized Replay	0,934	0,032

Deep Q-Learning présente un écart vingt fois supérieur à celui de Double DQN pour un niveau final identique. Selon la graine, il produit un agent excellent ou médiocre. À 100 000 épisodes cet écart disparaît : tous convergent.

L'apport du réseau cible n'est donc pas un meilleur plafond, mais une convergence plus rapide et beaucoup plus fiable. C'est exactement ce que la théorie annonce, et c'est invisible si l'on ne regarde que le score final.

6.4 La recherche sans apprentissage a une limite mesurable

MCTS et Random Rollout ne possèdent aucun réseau : ils simulent à chaque coup. Contre un adversaire aléatoire, ils atteignent +1,00 partout, sur les deux environnements à deux joueurs. Contre l'adversaire heuristique du Bobail, ils échouent.

Agent	vs aléatoire	vs heuristique	ms / coup
MCTS (UCT), 200 itérations	+1,00	-0,52	66,1

Random Rollout, 20 simulations	+1,00	-0,69	66,7
PPO	+1,00	+1,00	0,028
Double DQN + Replay	+0,99	+0,98	0,023

Deux enseignements. D'abord, un agent entraîné réussit là où la recherche échoue : PPO atteint +1,00 contre le même adversaire. Ensuite, le coût par décision est sans commune mesure — 66 millisecondes contre 0,028, soit un rapport de 2 400. Et ce coût est payé à chaque coup, indéfiniment, là où un agent entraîné le paie une fois pendant l'entraînement.

La formulation que nous retenons : la recherche sans apprentissage bat un adversaire faible et perd contre un adversaire compétent, tout en coûtant trois ordres de grandeur de plus par décision.

Nous n'affirmons pas que MCTS est intrinsèquement inférieur. Nous mesurons qu'à budget fixé — 200 itérations par coup — il ne voit pas assez loin pour ce jeu. Une mesure annexe le confirme : en portant les itérations à 2 000, MCTS passe de -0,60 à +1,00 contre un adversaire heuristique proche, au prix de 556 ms par coup. La difficulté tient donc à l'horizon de recherche, non à la méthode.

6.5 Le plafond de REINFORCE, et ce que le clipping y change

Sur TicTacToe, les trois variantes de REINFORCE restent entre 0,81 et 0,84 à un million d'épisodes, sans progression après 10 000. PPO, qui appartient à la même famille, atteint 0,98. Le seul mécanisme qui les sépare est le rapport clippé, qui borne le déplacement de la politique et autorise plusieurs passes sur le même lot.

Sur Bobail contre l'adversaire heuristique, l'écart est encore plus net : PPO obtient +1,00, REINFORCE +0,57 et REINFORCE + baseline +0,09. La baseline, qui réduit la variance sans biaiser le gradient, ne suffit pas quand la tâche devient difficile.

6.6 La longueur des parties révèle des stratégies que le score cache

Contre l'adversaire heuristique du Bobail, à 100 000 épisodes :

Agent	Score	Longueur moyenne
PPO	+1,00	9,7 coups
Double DQN + Replay	+0,98	12,1 coups
REINFORCE + critique	+0,86	22,2 coups
Deep Q-Learning	-0,63	38,4 coups
REINFORCE + baseline	+0,09	80,5 coups

REINFORCE + baseline obtient un score proche de zéro avec des parties huit fois plus longues que PPO. Il n'a pas appris à gagner : il a appris à faire durer, ce qui évite la défaite sans produire la victoire. Le score seul ne le dirait pas ; la longueur des parties le révèle.

Plus généralement, sur cet environnement la longueur des parties est inversement corrélée à la qualité de l'agent.

6.7 Instabilités observées

Nous rapportons deux comportements que des courbes lissées masqueraient.

Effondrement tardif. REINFORCE + baseline tient +1,00 sur Line World aux paliers 1 000, 10 000 et 100 000, puis chute à -0,33 à un million. Le détail par graine est éloquent : -1,0, +1,0, -1,0. Une graine sur trois conserve la politique optimale, deux divergent en fin d'entraînement. Avec une seule graine, nous aurions conclu au hasard.

Creux de milieu d'entraînement. Sur Bobail contre l'adversaire aléatoire, Double DQN + Replay et sa variante priorisée chutent à 0,71 et 0,60 au palier 10 000, avec des écarts entre graines de 0,54 et 0,99, avant de remonter à 0,99 et 0,97 à 100 000. Le phénomène n'apparaît ni avant ni après : il serait invisible avec deux paliers de mesure au lieu de quatre.

6.8 Une limite de notre propre mesure

Le temps par coup est quasiment identique pour les huit agents neuronaux : de 0,020 à 0,028 milliseconde. Ce n'est pas un résultat de comparaison entre algorithmes, mais la conséquence d'une architecture de réseau volontairement commune : on mesure la même passe avant. La métrique ne devient discriminante qu'en face des agents de planification, qui simulent.

Nous le signalons explicitement pour ne pas laisser croire que ces chiffres comparent des coûts algorithmiques.

7. Limites de l'étude

7.1 Le palier d'un million n'a pas été atteint partout

Le sujet mentionne ce palier « si possible ». Il a été atteint sur Line World et TicTacToe. Extrapolation depuis les débits mesurés en entraînement, pour huit agents apprenants et trois graines :

Environnement	1k + 10k + 100k	avec le palier 1M
Line World	0,3 h	2,7 h
Grid World	3,9 h	38,8 h
TicTacToe	0,3 h	3,5 h
Bobail	1,9 h	19,0 h
Total	6,4 h	64,0 h

Grid World à lui seul demanderait 38,8 heures, Bobail 19. Nous avons préféré une limite mesurée et argumentée à une case remplie approximativement.

7.2 Trois graines

Trois répétitions suffisent à détecter une instabilité — le cas de REINFORCE + baseline le montre — mais pas à établir une différence fine entre deux algorithmes proches. C'est une raison supplémentaire de ne pas conclure sur le classement final de 6.2.

7.3 L'adversaire heuristique n'est pas un joueur fort

Il applique trois règles sur le déplacement du bobail, et choisit son pion au hasard. Il suffit à discriminer les agents, mais un agent qui le bat n'a pas pour autant appris à jouer au Bobail. Nous mesurons une performance contre un adversaire donné, non une maîtrise du jeu.

Une vérification menée sur TicTacToe illustre le risque. Notre meilleur agent y gagne 99,3 % des parties, mais examen fait, il ignore 31,8 % des situations où l'adversaire menace de gagner au coup suivant : il attaque au lieu de se défendre. Contre un adversaire aléatoire cette stratégie est optimale, puisque la menace n'est presque jamais exécutée. L'agent n'a pas appris le morpion, il a appris à exploiter un adversaire aléatoire.

8. Conclusion

Onze algorithmes, quatre environnements, 165 entraînements et plus de 55 millions d'épisodes joués aboutissent à quatre conclusions que nous tenons pour établies.

- Trois des quatre environnements sont saturés. Seuls TicTacToe et Bobail contre un adversaire heuristique autorisent une comparaison.
- Sur une tâche simple, les quatre agents de la famille valeur et PPO convergent vers le même niveau. Le classement final relève du bruit ; ce qui les sépare est la vitesse et la fiabilité de la convergence, lisibles dans l'écart entre graines à mi-entraînement.
- La recherche sans apprentissage — MCTS, Random Rollout — atteint le score maximal contre un adversaire faible et échoue contre un adversaire compétent, pour un coût par décision 2 400 fois supérieur à celui d'un agent entraîné.
- Dans la famille gradient de politique, le rapport clippé de PPO est le mécanisme décisif : il fait passer de 0,82 à 0,98 sur TicTacToe, et de 0,57 à 1,00 sur Bobail contre l'adversaire heuristique.

Si nous devons poursuivre, deux directions nous paraissent prioritaires : porter le palier d'un million sur Bobail, dont le classement à 100 000 épisodes n'est peut-être pas stabilisé, et opposer nos agents entraînés à un adversaire de force croissante pour mesurer non plus une performance mais une généralisation.

Annexe — Reproduire les résultats

Deux environnements Python sont nécessaires : le cœur du projet ne dépend que de NumPy, les agents neuronaux exigent PyTorch, qui ne supporte pas encore Python 3.14.

```
python3 -m venv .venv && .venv/bin/pip install numpy pytest pygame-ce
python3.12 -m venv .venv-torch && .venv-torch/bin/pip install -r requirements.txt
```

Vérification de l'installation — 30 tests unitaires portant sur le contrat des environnements et sur les règles du Bobail :

```
.venv-torch/bin/python -m pytest tests -q
```

Les trois campagnes de ce rapport :

```
python -m src.experiment --envs line_world,tictactoe --seeds 0,1,2 --episodes 1 000 000 --eval-episodes 200
python -m src.experiment --envs grid_world,bobail --seeds 0,1,2 --episodes 100 000 --eval-games 200
python -m src.experiment --envs bobail --opponent heuristic --seeds 0,1,2 --episodes 100 000 --eval-games 200 --out
runs_heuristic
```

Tableaux de synthèse et figures :

```
python -m src.report_figures
```

Rejouer un modèle sauvegardé :

```
python -m src.eval --env bobail --agent ppo --model models_livrables/bobail_vs_heuristique__ppo__100 000ep.pt --games
500
```

Interface graphique — observer un agent, ou jouer contre l'environnement :

```
python -m src.gui.app --env bobail --agent mcts_uct
python -m src.gui.app --env bobail --agent human
```